

Day05 Part D 学习文档 v1.0 : Tool Registry (Capability Management Center)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part D 正式学习文档。

Day05 Part A 建立 Tool Calling 的基础模型，Part B 解释 LLM 如何做 Tool Decision，Part C 说明 Tool Schema 如何定义单个 Tool 的行动契约。Part D 继续回答一个更工程化的问题：当 Runtime 拥有大量 Tool 时，如何注册、组织、筛选、路由和暴露这些能力？

本节定位

Day05 的主题是 Tool Calling，也就是 Agent Runtime 的 Execution Engine。

Part A 回答：

Tool Calling 是什么？

核心是：

```
LLM
|
| Tool Call Intent
v
Runtime
|
| validate + execute
v
Tool
|
| Observation
v
Runtime State
```

Part B 回答：

LLM 为什么会决定调用 Tool？

核心是：

Goal-driven Action Selection

Part C 回答：

Runtime 如何描述一个 Tool，让 LLM 能正确理解、选择、填参和解释结果？

核心是：

Tool Schema = Action Contract

Part D 继续回答：

Runtime 如何管理大量 Tool，并决定当前这一轮应该给 LLM 看哪些 Tool？

核心是：

Tool Registry = Capability Management Center

本节的核心结论是：

Tool Registry 不是一个 Map<String, Tool>，而是 Agent Runtime 的能力管理中心。它负责 Tool 的注册、发现、分类、权限、生命周期、路由候选集生成，以及向 Context Builder 提供当前可用的 Tool Schema。它管理的是 Agent 的 Action Space。

目录

- 与 Part C 的联系
- 为什么需要 Tool Registry
- Tool Registry 是什么
- Registry 管理所有能力，LLM 只看到当前能力
- Tool Definition 与 Tool Implementation 分离
- Tool Registration
- Tool Discovery
- Static Registry 与 Dynamic Registry
- Tool Routing
- Tool Registry 与 Tool Router 的边界
- Registry 与 Context Builder 的关系
- Tool Metadata
- Permission 与 Capability Security
- Lifecycle 与 Versioning
- Tool Registry Evaluation
- MCP 与 Tool Registry
- Tool 与 MCP 的区别
- 内置 Tool 与外部 MCP 如何选择
- Agent 设计与传统软件工程的关系
- mini-agent-runtime 中的最小数据模型
- mini-agent-runtime 最小实现草图
- 工业术语映射
- 面试视角
- 写书 TODO
- 写书素材
- 本 Part 核心认知升级
- 下一节学习计划
- 本章思考题

与 Part C 的联系

Part C 讨论的是单个 Tool :

```
Tool Schema
|
| name
```

```
| description
| parameters
| outputSchema
| errorSchema
| metadata
v
Action Contract
```

它回答：

如何把一个能力描述给 LLM？

但真实 Agent Runtime 不会只有一个 Tool。

一个 Coding Agent 可能有：

```
read_file
write_file
search_files
execute_command
apply_patch
git_status
git_diff
git_commit
open_browser
click
inspect_dom
...
```

一个企业客服 Agent 可能有：

```
query_order
query_refund
create_refund_request
check_inventory
query_logistics
send_email
create_ticket
handoff_to_human
...
```

一个接入 MCP 的 Agent 可能动态发现几十个甚至上百个外部能力。

所以 Part C 结束后自然出现一个新问题：

如果 Runtime 拥有很多 Tool，是否应该把所有 Tool Schema 每一轮都塞给 LLM？

答案是：

不应该。

因为 Tool 越多，Action Space 越大，模型选择难度越高，Context 成本越高，误调用概率也越高。

因此 Part D 进入：

```
Single Tool Design
|
v
Tool Schema
|
v
Many Tools Management
|
v
Tool Registry
```

Part C 研究的是：

如何设计单个 Action。

Part D 研究的是：

如何管理整个 Action Space。

为什么需要 Tool Registry

最简单的 Agent 可以这样写：

```
const tools = {  
  search_files,  
  read_file,  
  write_file,  
};
```

然后每次调用 LLM 时，把这些 Tool 全部传进去。

当 Tool 数量很少时，这可以工作。

但随着 Agent 工程复杂度上升，会出现几个问题。

问题一：Action Space 爆炸

假设 Runtime 有 100 个 Tool。

用户说：

帮我总结一下 README。

这一轮真正可能需要的 Tool 也许只有：

```
read_file  
search_files
```

但如果 Runtime 暴露全部 Tool，LLM 会看到：

```
read_file  
write_file  
delete_file  
execute_command  
send_email  
query_database  
create_ticket  
refund_order  
...
```

这会带来三个问题：

模型需要在更大的候选空间里选择；

无关 Tool 占用 Context；

高风险 Tool 被不必要地暴露给模型。

Tool 越多，不一定 Agent 越强。

更准确地说：

```
更多 Tool  
!=  
更强 Agent
```

只有当 Runtime 能正确管理、筛选和暴露 Tool 时，更多 Tool 才可能变成更强能力。

问题二：Tool 来源复杂

Tool 不一定都写在 Agent 项目内部。

它们可能来自：

- Internal Function
- Internal Service
- Database
- HTTP API
- Plugin
- MCP Server
- Remote Capability Provider

Runtime 需要一个统一入口来管理这些能力。

问题三：Tool 不是永远可用

某些 Tool 只在特定条件下可用：

- 用户已登录
- 当前 workspace 有写权限
- 当前任务允许联网
- 当前项目是 Git 仓库
- 当前环境配置了 GitHub MCP
- 当前用户有 refund 权限

所以 Tool Availability 是动态的。

Runtime 需要知道：

- 所有注册能力
- 当前可用能力
- 当前应该暴露给 LLM 的能力
- 当前允许执行的能力

这些不是一个普通数组能解决的。

问题四：权限和风险需要集中控制

Tool 有风险等级：

- read-only
- write
- network
- payment
- delete
- external_send
- human_approval_required

例如：

- read_file

风险很低。

但：

- delete_file
- send_email
- refund_order
- execute_shell

就需要权限判断、确认或审批。

如果 Tool 只是零散函数，Runtime 很难统一做安全治理。

Tool Registry 是什么

最简单定义：

Tool Registry 是 Runtime 中管理所有可用 Tool 的注册表。

但这只是表层。

更准确的定义是：

Tool Registry 是 Agent Runtime 的 Capability Management Center，负责管理 Agent 能力的定义、来源、分类、权限、可用性、路由和暴露策略。

它管理的不只是函数。

它管理的是：

Capability

也就是 Agent 可以做什么。

因此 Tool Registry 的核心职责包括：

注册 Tool ;

保存 Tool Definition ;

绑定 Tool Implementation ;

维护 Tool Metadata ;

按任务、权限、上下文筛选 Tool ;

支持静态和动态 Tool Discovery ;

为 Context Builder 提供当前可见 Tool Schema ;

为 Tool Executor 提供可执行实现；

支持 Tool 生命周期、版本和测试。

可以这样理解：

```

Tool Registry
|
+-- Capability Catalog
+-- Tool Definition Store
+-- Tool Implementation Binding
+-- Tool Metadata Store
+-- Tool Availability Policy
+-- Tool Routing Candidate Provider
+-- Context Builder Input Source

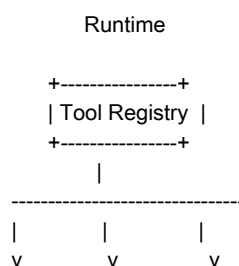
```

Registry 管理所有能力，LLM 只看到当前能力

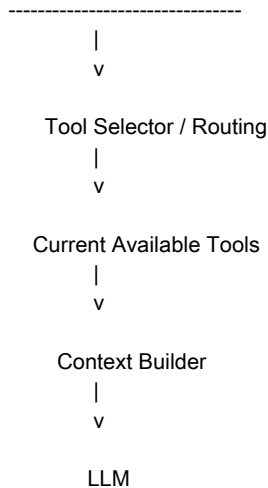
这一点是 Part D 的核心。

Registry 管的是所有能力。
LLM 每轮只看到当前能力。

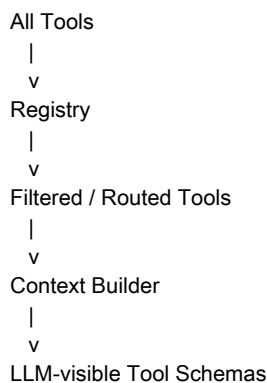
完整链路：



Search Tool Order Tool Email Tool
Read Tool Refund Tool Calendar Tool



这意味着：



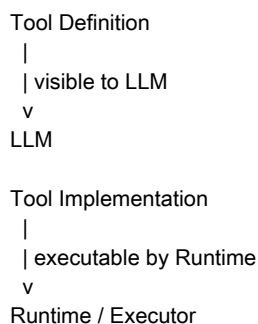
LLM 不需要知道所有能力。

LLM 只需要在当前任务中看到足够相关、足够安全、足够清晰的能力。

Tool Definition 与 Tool Implementation 分离

Part C 已经讲过 Tool Schema 是 Action Contract。

Part D 需要把它放进 Runtime 结构中：



Tool Definition

Tool Definition 是 LLM 看得见的能力描述：

```
type ToolDefinition = {
  name: string;
  description: string;
```

```
inputSchema: JSONSchema;
outputSchema?: JSONSchema;
errorSchema?: JSONSchema;
};
```

它回答：

这个 Tool 是什么？
什么时候用？
需要什么参数？
返回什么结果？
失败时代表什么？

Tool Implementation

Tool Implementation 是 Runtime 真正执行的函数：

```
type ToolImplementation = (input: unknown, ctx: ToolExecutionContext) => Promise<ToolResult>;
```

它回答：

怎么执行？
调用哪个 API？
查哪个数据库？
需要什么凭证？
失败怎么抛错？

为什么要分离

原因一：同一个 Definition 可以对应不同实现。

```
query_order_status
|
+-- dev mock implementation
+-- staging API implementation
+-- production API implementation
```

原因二：LLM 不应该看到 Runtime 内部细节。

LLM 需要知道：

查询订单状态

但不应该知道：

数据库连接串
内部服务地址
认证 token
重试策略
低层 SDK 调用细节

原因三：权限控制需要 Runtime 处理。

LLM 可以生成：

refund_order

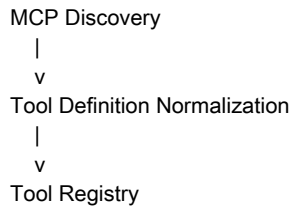
但 Runtime 必须决定：

当前用户是否允许退款？
是否需要 human approval？
是否超过金额阈值？

原因四：MCP 动态发现需要统一归一化。

MCP Server 暴露的外部 Tool 最终也需要进入 Runtime Registry：

```
MCP Server
|
v
```

Tool Registration

Tool Registration 是把能力注册进 Registry 的过程。

最小注册结构：

```
registry.register({
  definition: {
    name: "read_file",
    description: "Read a text file from the workspace.",
    inputSchema: {
      type: "object",
      properties: {
        path: {
          type: "string",
          description: "Workspace-relative file path.",
        },
      },
      required: ["path"],
    },
  },
  metadata: {
    category: "filesystem",
    riskLevel: "read",
    source: "internal",
  },
  execute: async (input, ctx) => {
    return ctx.fs.readText(input.path);
  },
});
```

注册时 Runtime 至少要检查：

Tool name 是否唯一；

inputSchema 是否合法；

implementation 是否存在；

metadata 是否完整；

权限和风险等级是否可识别；

是否允许覆盖已有版本。

因此 Tool Registration 不是简单 push：

```
tools.push(tool);
```

而是能力契约进入 Runtime 的入口。

Tool Discovery

Tool Discovery 是 Runtime 发现可用能力的过程。

它可以是静态的：

```
import { readFileTool } from "../tools/read-file";
import { searchFilesTool } from "../tools/search-files";

registry.register(readFileTool);
registry.register(searchFilesTool);
```

也可以是动态的：

```
Runtime 启动
|
v
连接 MCP Server
|
v
list tools
|
v
转换成 Runtime Tool Definition
|
v
注册到 Tool Registry
```

例如：

```
GitHub MCP Server
|
+-- search_repositories
+-- create_issue
+-- get_pull_request
```

进入 Runtime 后会变成：

```
Tool Registry
|
+-- github.search_repositories
+-- github.create_issue
+-- github.get_pull_request
```

Tool Discovery 的意义是：

Agent 不需要在代码里提前写死所有外部能力，而是可以通过协议、插件或配置动态发现能力。

Static Registry 与 Dynamic Registry

Static Registry

Static Registry 指 Tool 在代码中固定注册。

适合：

- 核心业务能力
- 内置文件操作
- 稳定内部 API
- 固定工作流动作

优点：

- 简单；
- 可控；
- 易测试；
- 安全边界清晰。

缺点：

扩展需要改代码；
外部系统接入成本高；
不适合插件化生态。

Dynamic Registry

Dynamic Registry 指 Runtime 在运行时发现和注册 Tool。

适合：

- MCP
- 插件系统
- 第三方集成
- 多租户企业能力
- 按 workspace 动态加载能力

优点：

- 可扩展；
- 可插件化；
- 适合多 Agent 复用；
- 适合外部能力生态。

缺点：

- 需要更强权限控制；
- 需要版本和兼容性管理；
- 需要动态能力测试；
- 需要处理发现失败、连接失败和能力变更。

工业 Agent 往往两者都需要：

- Internal Core Tools -> Static Registry
- External Integrations -> Dynamic Registry

Tool Routing

Tool Routing 是 Registry 的核心升级。

如果 Registry 只是保存全部 Tool，它还没有解决 Action Space 爆炸。

真正关键是：

当前任务到底应该暴露哪些 Tool？

这就是 Tool Routing。

两阶段选择

工业 Agent 常采用两阶段：

```
Stage 1: Tool Retrieval / Routing
|
| 从大量 Tool 中筛出候选集
v
Candidate Tools
|
v
Stage 2: LLM Tool Decision
```

|
| 在候选 Tool 中选择是否调用、调用哪个、参数是什么
v
Tool Call Intent

例如：

All Tools: 200
|
| routing
v
Candidate Tools: 5
|
| LLM decision
v
Selected Tool: read_file

Tool Routing 不一定依赖 LLM

Tool Router 可以有多种实现。

方法一：规则 Routing

根据任务类型、权限、环境进行规则筛选：

```
if (task.kind === "code_review") {  
  return registry.byCategory(["filesystem", "git"]);  
}  
  
if (task.kind === "customer_support") {  
  return registry.byCategory(["order", "refund", "ticket"]);  
}
```

优点是稳定、便宜、可解释。

方法二：Embedding Routing

把 Tool description 做 embedding，再根据用户任务检索相关 Tool：

User Goal Embedding
|
v
Vector Search over Tool Descriptions
|
v
Top-K Candidate Tools

优点是适合大量 Tool。

方法三：LLM Router

先让一个轻量 LLM 判断任务类型或候选能力：

User Goal
|
v
Router LLM
|
v
Candidate Tool Categories

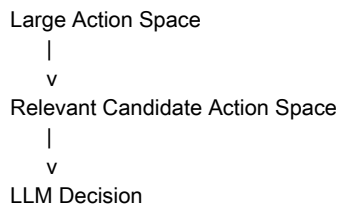
优点是语义理解强。

缺点是成本更高，也需要评估和防错。

Routing 的目标

Tool Routing 的目标不是替 LLM 完成最终 Tool Call。

它的目标是缩小 Action Space：



Tool Registry 与 Tool Router 的边界

这两个概念容易混在一起。

Tool Registry

Tool Registry 是能力目录：

- 有哪些 Tool？
- 它们的定义是什么？
- 来源是什么？
- 属于什么类别？
- 风险等级是什么？
- 是否可用？
- 如何执行？

它更像：

Capability Catalog

Tool Router

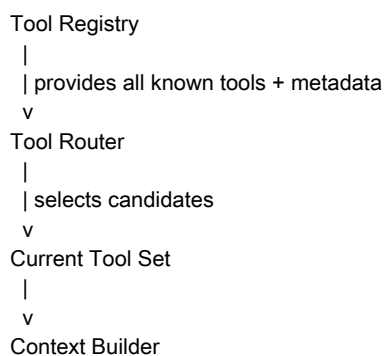
Tool Router 是候选能力选择器：

- 当前任务可能需要哪些 Tool？
- 当前上下文应该暴露哪些 Tool？
- 哪些 Tool 应该被过滤掉？

它更像：

Capability Retriever / Action Retriever

两者关系：



简单说：

- Registry 管所有能力。
- Router 选当前候选能力。

Registry 与 Context Builder 的关系

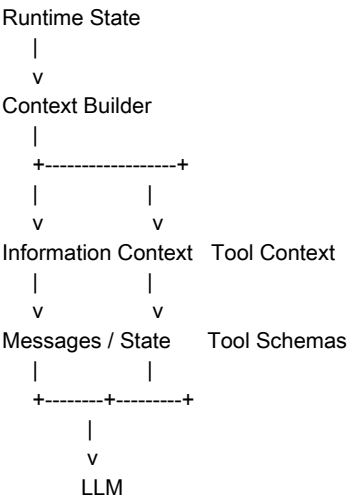
Day04 的 Context Builder 决定：

LLM 看到什么信息

Day05 的 Tool Registry 决定：

LLM 看到什么能力

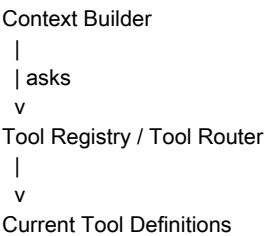
二者共同决定 LLM 当前的工作面：



Tool Registry 是 Context Builder 的输入源之一。

Context Builder 不应该自己硬编码 Tool 列表。

更合理的结构是：



这也是 Tool-aware Context Engineering：

Context Engineering 管理信息空间，Tool Registry 管理行动空间。

Tool Metadata

Tool Definition 是 LLM 世界。

Tool Metadata 是 Runtime 世界。

```
type ToolMetadata = {
  id: string;
  name: string;
  category: ToolCategory;
  source: ToolSource;
  riskLevel: ToolRiskLevel;
  permission: PermissionPolicy;
  version: string;
  enabled: boolean;
  tags?: string[];
  owner?: string;
```

```
};
```

Metadata 不一定暴露给 LLM。

它主要给 Runtime 使用：

- 权限判断
- 风险控制
- 路由筛选
- 生命周期管理
- 版本兼容
- 审计日志
- 评估测试

例如：

```
{
  name: "send_email",
  category: "communication",
  source: "internal",
  riskLevel: "external_write",
  permission: {
    requiresApproval: true,
    scopes: ["email:send"]
  },
  version: "1.2.0",
  enabled: true
}
```

LLM 需要看到的是：

send_email 可以发送邮件，需要 recipient、subject、body。

Runtime 需要知道的是：

这个 Tool 会对外发送信息，需要审批，需要 email:send scope，要写审计日志。

这就是：

Capability 与 Control 分离

Permission 与 Capability Security

Tool Registry 是 Permission 系统的前置入口。

它至少需要描述：

- Tool 是否可见
- Tool 是否可调用
- Tool 是否需要审批
- Tool 是否只读
- Tool 是否会产生外部副作用
- Tool 是否允许当前用户使用

一个常见风险分级：

```
type ToolRiskLevel =
  | "read"
  | "write"
  | "network"
  | "external_send"
  | "delete"
  | "payment"
  | "system";
```

不同风险等级对应不同 Runtime Policy：

read

-> 通常可直接执行

write

-> 可能需要 workspace 权限

external_send

-> 发送前需要确认

delete

-> 需要强确认或禁止自动执行

payment

-> 需要 human approval

system

-> 严格 sandbox 或默认禁用

Tool Registry 不一定完成最终审批，但它需要提供足够的 metadata，让后续 Permission / Human Approval 系统能做判断。

Lifecycle 与 Versioning

Tool Registry 需要生命周期管理。

因为 Tool 会变化：

新增 Tool

下线 Tool

禁用 Tool

升级 inputSchema

修改 outputSchema

调整 description

替换 implementation

改变 permission policy

这些变化都会影响 Agent 行为。

尤其是：

Tool Description 变化

可能改变 LLM 的 Tool Selection。

所以工业 Registry 需要：

enabled / disabled ；

deprecated ；

version ；

owner ；

changelog ；

compatibility policy ；

regression evaluation。

示例：

```
type ToolLifecycle = {  
  status: "enabled" | "disabled" | "deprecated";  
  version: string;  
  deprecatedAt?: string;  
  replacement?: string;
```



```
    changelog?: string;
};
```

Tool Registry 不是静态配置文件，而是 Agent 能力生命周期的管理系统。

Tool Registry Evaluation

Tool Registry 变化需要测试。

原因是：

Tool 列表、Tool Schema、Tool Metadata 和 Tool Routing 规则都会改变模型行为。

常见测试包括：

Tool Selection Evaluation

给定用户任务，检查候选 Tool 是否正确：

```
Input: "帮我查看订单 ORD-10001 的物流"
Expected candidate tools:
- query_order
- query_logistics
Not expected:
- refund_order
- send_email
```

Capability Regression Testing

当新增或修改 Tool 后，检查老任务是否仍然选择正确工具。

```
新增 create_refund_request 后：
原来 query_refund_status 的任务不应该误调用 create_refund_request。
```

Permission Evaluation

检查高风险 Tool 是否被正确过滤或标记审批。

```
普通用户不应看到 refund_order。
大额退款必须 requiresApproval。
```

Context Budget Evaluation

检查每轮暴露的 Tool 数量是否合理。

```
Top-K candidate tools <= 8
无关工具不进入 Context
高风险工具默认不暴露
```

这说明 Tool Registry 已经不是普通工程配置，而是 Agent 行为稳定性的一部分。

MCP 与 Tool Registry

MCP 经常被误解成：

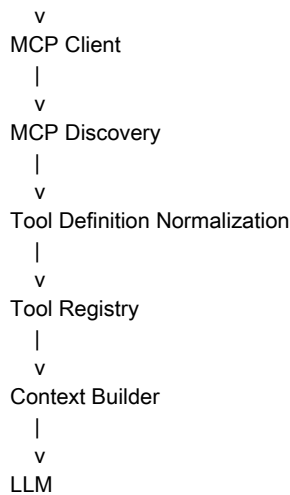
MCP = 让 Agent 调工具

更准确地说：

MCP 是一种标准化外部能力发现和通信协议，让外部系统可以把自己的能力暴露给 Agent Runtime。

在 Runtime 内部，MCP 暴露出的能力通常仍然要进入 Tool Registry。

```
MCP Server
|
```



因此：

MCP 不是替代 Tool Registry。
MCP 是产生外部 Tool 的方式。

Tool Registry 不关心能力来源。

它看到的是统一的 Capability：

```
Tool Registry
|
+-- read_file
|   source: internal
|
+-- query_order
|   source: internal_api
|
+-- github.create_issue
|   source: mcp
|
+-- calendar.create_event
|   source: mcp
```

MCP 解决的是：

外部能力如何被发现、描述、连接和调用。

Tool Registry 解决的是：

Runtime 如何统一管理这些能力，并决定哪些能力暴露给 LLM。

Tool 与 MCP 的区别

Tool

Tool 是 Agent 使用能力的抽象。

它可以是内部函数：

```
read_file
search_files
query_order_status
calculate_discount
```

也可以是外部系统能力被 Runtime 包装后的动作。

从 LLM 视角看，Tool 是：

```
{
```

```
"name": "query_order_status",
"description": "Query the status of an order by order id."
}
```

MCP

MCP 是连接外部能力的一种标准协议。

例如：

```
GitHub MCP Server
|
+-- search_repositories
+-- create_issue
+-- get_pull_request
```

Agent Runtime 通过 MCP Client 发现这些能力，并把它们纳入 Registry。

LLM 眼里的区别

对 LLM 来说，内部 Tool 和 MCP Tool 几乎没有区别。

LLM 看到的都是：

```
name
description
parameters
```

它并不知道这个 Tool 背后是：

```
内部函数
内部 HTTP API
MCP Server
第三方 SaaS
```

这个区别由 Runtime 管。

因此一句话总结：

Tool 是 Agent 使用能力的抽象，MCP 是让外部能力以标准方式成为 Tool 的协议。

内置 Tool 与外部 MCP 如何选择

判断标准不是技术，而是 Ownership。

问一个问题：

谁拥有这个能力？

适合内置 Tool 的能力

如果能力属于你的 Agent 或你的业务系统，通常适合做内置 Tool。

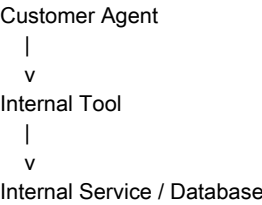
例如客服 Agent：

```
query_order_status
check_customer_level
calculate_discount
create_refund_request
query_product_inventory
```

这些是业务核心能力。

它们通常由你的团队控制权限、数据模型、业务规则和稳定性。

结构是：



适合 MCP 的能力

如果能力属于外部系统，并且希望被多个 Agent 复用，通常适合 MCP。

例如：

- GitHub
- Slack
- Google Drive
- Calendar
- Jira
- Notion
- Database Connector

这些能力有几个特点：

- 外部系统拥有；
- 多个 Agent 可能复用；
- 有独立生命周期；
- 不希望每个 Agent 都重写一遍；
- 适合通过协议动态发现。

判断表

问题	内置 Tool	MCP	---	---	---	能力属于 Agent 自己吗？	是	否	是否是业务核心逻辑？	通常是
通常否	是否访问内部数据库或内部服务？	通常是	可能	是否属于第三方系统？	可能	通常是				
是否需要多个 Agent 复用？	较少	通常是		是否希望插件化接入？	较少	通常是				
是否需要运行时动态发现？	较少	通常是		是否需要跨语言、跨进程、跨系统？	较少	通常是				

Agent 设计与传统软件工程的关系

学习到 Tool Registry、Tool Router、Permission、Workflow、Memory、State 之后，会发现一个规律：

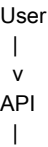
Agent 并没有推翻软件工程，而是把传统软件工程中的确定性系统设计与 LLM 的概率性决策结合起来。

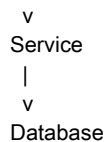
很多 Agent 概念都有传统软件工程映射：

Agent 概念	传统软件工程对应	---	---	Tool Registry	Service Registry / API Gateway	Tool Router
Request Routing / Retrieval	Workflow	Workflow Engine	Runtime State	Application State	Memory	
Database / Cache / Retrieval System	Executor	Task Executor	Permission	RBAC / Policy Engine		
Retry	Distributed System Reliability	Context Builder	Data Aggregation / Projection Layer	MCP		
Protocol / Integration Standard						

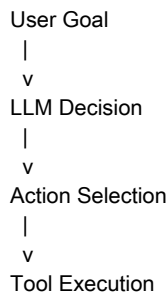
区别在于：

传统系统的执行路径通常提前写死：

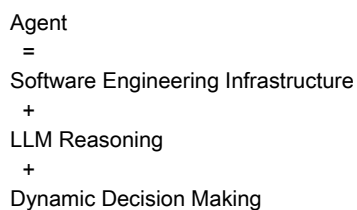




Agent 系统增加了动态决策层：



因此：



优秀 Agent 工程师本质上仍然需要优秀的软件工程能力，只是要额外理解：

如何把确定性工程系统暴露给概率性模型使用。

mini-agent-runtime 中的最小数据模型

为了实现一个最小但结构正确的 Tool Registry，可以先设计以下类型。

ToolDefinition

```
export type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: JSONSchema;
  outputSchema?: JSONSchema;
  errorSchema?: JSONSchema;
};
```

ToolMetadata

```
export type ToolSource = "internal" | "internal_api" | "mcp" | "plugin";

export type ToolCategory =
  | "filesystem"
  | "git"
  | "browser"
  | "search"
  | "business"
  | "communication"
  | "calendar"
  | "unknown";

export type ToolRiskLevel =
  | "read"
  | "write"
  | "network"
  | "external_send"
```

```

| "delete"
| "payment"
| "system";

export type ToolMetadata = {
  id: string;
  source: ToolSource;
  category: ToolCategory;
  riskLevel: ToolRiskLevel;
  version: string;
  enabled: boolean;
  tags?: string[];
  requiresApproval?: boolean;
};

```

RuntimeTool

```

export type RuntimeTool = {
  definition: ToolDefinition;
  metadata: ToolMetadata;
  execute: ToolImplementation;
};

```

ToolImplementation

```

export type ToolImplementation = (
  input: unknown,
  context: ToolExecutionContext
) => Promise<ToolResult>;

```

ToolResult

```

export type ToolResult =
| {
  ok: true;
  output: unknown;
}
| {
  ok: false;
  error: {
    code: string;
    message: string;
    retryable?: boolean;
  };
};

```

mini-agent-runtime 最小实现草图

一个最小 Registry 可以这样设计：

```

export class ToolRegistry {
  private tools = new Map<string, RuntimeTool>();

  register(tool: RuntimeTool): void {
    const name = tool.definition.name;

    if (this.tools.has(name)) {
      throw new Error(`Tool already registered: ${name}`);
    }

    this.tools.set(name, tool);
  }

  get(name: string): RuntimeTool | undefined {
    return this.tools.get(name);
  }
}

```

```

}

list(): RuntimeTool[] {
  return [...this.tools.values()];
}

listDefinitions(): ToolDefinition[] {
  return this.list()
    .filter((tool) => tool.metadata.enabled)
    .map((tool) => tool.definition);
}

selectCandidates(query: ToolCandidateQuery): RuntimeTool[] {
  return this.list().filter((tool) => {
    if (!tool.metadata.enabled) return false;
    if (query.categories && !query.categories.includes(tool.metadata.category)) {
      return false;
    }
    if (query.maxRisk && !isRiskAllowed(tool.metadata.riskLevel, query.maxRisk)) {
      return false;
    }
    return true;
  });
}

```

最小查询结构：

```

export type ToolCandidateQuery = {
  categories?: ToolCategory[];
  maxRisk?: ToolRiskLevel;
  tags?: string[];
};

```

Runtime Loop 中使用：

```

const candidateTools = toolRegistry.selectCandidates({
  categories: inferCategories(userGoal),
  maxRisk: runtimePolicy.maxRisk,
});

const context = contextBuilder.build({
  state,
  tools: candidateTools.map((tool) => tool.definition),
});

const response = await llm.generate(context);

```

然后执行时：

```

const tool = toolRegistry.get(toolCall.name);

if (!tool) {
  throw new Error(`Unknown tool: ${toolCall.name}`);
}

await permissionPolicy.assertCanExecute(tool, runtimeContext);

const result = await tool.execute(toolCall.arguments, runtimeContext);

```

这样最小实现已经具备正确分层：

```

Registry
-> 管理能力

Context Builder
-> 暴露能力描述

```

LLM

-> 生成 Tool Call Intent

Permission Policy

-> 决定是否允许执行

Executor

-> 执行 Tool Implementation

工业术语映射

| 本章术语 | 工业术语 || --- | --- || Tool Registry | Capability Registry || Tool Definition | Action Contract / Function Definition || Tool Implementation | Handler / Executor Binding || Tool Metadata | Runtime Control Metadata || Tool Routing | Capability Retrieval / Action Retrieval || Current Tools | Candidate Tool Set || MCP Discovery | Dynamic Capability Discovery || Tool Category | Capability Taxonomy || Permission Policy | Capability Security Policy || Tool Evaluation | Capability Regression Testing |

面试视角

Q1 : Tool Registry 和简单工具列表有什么区别？

简单工具列表只是保存可调用函数。

工业 Agent 中 Tool Registry

是能力管理中心，负责工具注册、发现、分类、权限控制、生命周期、路由候选集生成，以及向 Context Builder 提供当前可用 Tool Schema。

Q2 : 为什么不能把所有 Tool 都给 LLM？

因为 Tool 数量增加会扩大 Action Space，导致模型决策困难、上下文浪费和错误调用概率增加。工业 Agent 通常通过 Registry 和 Routing 机制先筛选候选工具，再交给 LLM 做最终 Tool Decision。

Q3 : Tool Registry 和 Context Builder 是什么关系？

Context Builder 决定 LLM 看见什么信息。

Tool Registry 决定 LLM 可以使用什么能力。

Registry 提供经过筛选的 Tool Schema，Context Builder 将其组合进最终 Context。

Q4 : Tool Registry 和 Tool Router 的边界是什么？

Tool Registry 是能力目录，管理所有 Tool 的定义、实现、metadata 和状态。

Tool Router 是候选能力选择器，负责根据当前任务从 Registry 中筛出相关 Tool。

Q5 : MCP 和 Tool Registry 的关系是什么？

MCP 负责标准化外部能力发现和通信。

Tool Registry 负责 Runtime 内部能力管理。

MCP Server 暴露的工具最终通常需要进入 Runtime Registry，才能被 Agent 统一筛选、暴露和执行。

Q6 : Tool 和 MCP 的区别是什么？

Tool 是 Agent 使用能力的抽象。

MCP 是让外部能力以标准方式成为 Tool 的协议。

对 LLM 来说，内部 Tool 和 MCP Tool 都是 Tool Schema；对 Runtime 来说，它们的来源、权限、连接方式和生命周期不同。

写书 TODO

后续整理成正式书稿时，本章可以作为：

Chapter: Tool Registry - Managing Agent Capabilities

建议写作结构：

- 从 Tool List 到 Capability Management；
- Tool Registry 为什么不是简单 Map；
- Tool Definition 与 Tool Implementation 分离；
- Tool Metadata：Runtime 控制信息；
- Context Engineering 与 Capability Engineering 的关系；
- Tool Router 独立成小节，讲 Action Space 缩小；
- MCP 作为 Dynamic Capability Discovery；
- Permission、Lifecycle、Evaluation 作为工业化补充；
- mini-agent-runtime 最小实现。

写作时要避免把 Registry 写成纯数据结构章节。

本章真正要表达的是：

Agent Runtime 不只是调用工具，而是在管理模型可以进入的行动空间。

写书素材

素材 1：Tool Registry 核心定义

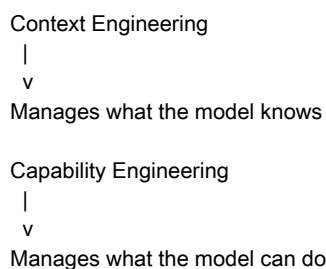
英文版：

A Tool Registry is not just a collection of tools. It is the capability management layer of an Agent Runtime, responsible for discovering, organizing, filtering, securing, and exposing available capabilities to the model.

中文版：

Tool Registry 不只是工具集合，而是 Agent Runtime 的能力管理层，负责发现、组织、筛选、安全控制以及向模型暴露可用能力。

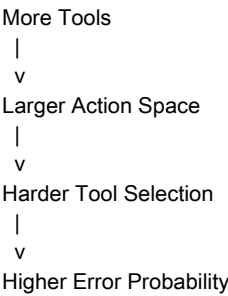
素材 2：Context 与 Capability 的关系



一句话：

Context Engineering 管理信息空间，Tool Registry 管理行动空间。

素材 3：为什么 Tool 越多不一定越强



要让 Tool 数量变成能力优势，需要 Registry、Routing、Permission 和 Evaluation。

素材 4：Tool Registry 与 MCP



一句话：

MCP 不是替代 Tool，而是让外部能力以标准方式进入 Tool Registry。

本 Part 核心认知升级

完成 Part D 后，认知应从：

Tool Registry = 保存工具的 Map

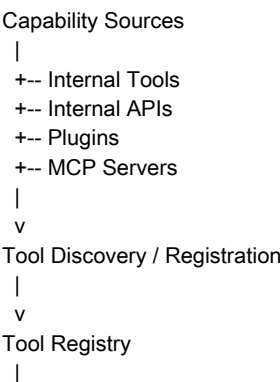
升级为：

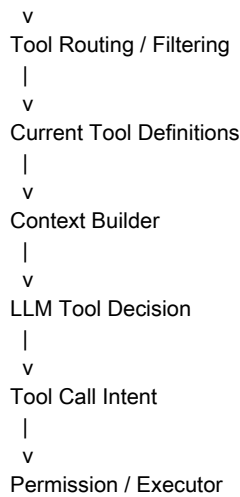
Tool Registry = Agent Runtime 的 Capability Management Center

再进一步：

Tool Registry = 管理 Agent Action Space 的核心系统

完整链路：





本 Part 的最终结论：

Agent 的能力不是由 Tool 数量直接决定的，而是由 Runtime 如何管理、筛选、约束和暴露这些 Tool 决定的。

下一节学习计划

Day05 Part E：Tool Executor。

Part D 解决的是：

Runtime 如何管理和暴露 Tool？

Part E 继续解决：

Runtime 如何安全、可靠、可观测地执行 Tool Call Intent？

重点问题：

Tool Executor 是什么；

LLM 为什么不能直接执行 Tool；

Tool Call Intent 如何被 Runtime 校验；

Tool 参数如何 validate；

Tool Execution Context 如何设计；

Tool 执行失败如何返回 Error Observation；

Retry、Timeout、Cancellation 如何处理；

Permission & Human Approval 如何接入执行链路；

Tool Result 如何回流 Runtime State；

mini-agent-runtime 中如何实现最小 Tool Executor。

本章思考题

为什么 Tool Registry 不是简单的 Map<String, Tool>？

Tool Registry 和 Tool Definition 有什么区别？

为什么 Tool Definition 与 Tool Implementation 要分离？

Tool Metadata 应该包含哪些 Runtime 控制信息？

为什么不能把所有 Tool 都暴露给 LLM？

Tool Routing 如何降低 Action Space？

Tool Router 是否一定需要 LLM？

Static Registry 和 Dynamic Registry 分别适合什么场景？

MCP 暴露的 Tool 为什么仍然需要进入 Runtime Registry？

Tool Registry 和 Context Builder 的关系是什么？

Context Engineering 和 Capability Engineering 有什么区别？

Tool Registry 如何连接 Permission & Human Approval？

为什么 Tool Registry 变化需要 Regression Testing？

Tool Description 修改为什么可能改变 Agent 行为？

如何判断一个能力应该做成内置 Tool 还是 MCP？

从 Ownership 角度如何区分 Tool 和 MCP？

为什么说 Agent 没有推翻软件工程，而是扩展了软件工程？

Tool Registry 在企业 Agent 平台中对应哪些传统工程系统？

mini-agent-runtime 的最小 Tool Registry 至少应该有哪些方法？

Tool Registry 与下一节 Tool Executor 如何衔接？

Source

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a69b440-0350-83e8-ae8b-6671a4295440>
- 本地源记录：source/day05-part-d-chatgpt-share-source.md